MINISTÉRIO DA EDUCAÇÃO
SECRETARIA DE EDUCAÇÃO PROFISSIONAL E TECNOLÓGICA
INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DO CEARÁ
CAMPUS MARANGUAPE

Curso Técnico em Informática para Internet
Disciplina: Desenvolvimento Web 2 · Prof. Kaio Jonathas

Aluno(a): Turma: Data: / /

Lista de Exercícios: JavaScript (Aula 1)

1. Como usar esta lista

São 77 questões que seguem a ordem dos slides, da instalação do Node.js até arrays e objetos; classes e POO ficam para outra lista. Dica: **preveja a resposta antes de executar** o código, pois é aí que a aprendizagem acontece.

- **Numeração:** a questão 6.3 é a terceira da seção 6.
- **Nível:** (B) básico, (M) médio, (D) desafio.
- **Como rodar:** salve o código em um arquivo `.js` e execute `node arquivo.js` no terminal do VS Code.
- **Seção 10:** exige `npm install prompt-sync` na pasta do projeto.
- **Questões "o que imprime?":** anote a previsão no caderno e só depois confira no terminal.

2. Ambiente, Node.js e Hello World

2.1 (B) Associe cada tecnologia ao seu papel em uma página web: HTML, CSS e JavaScript. As opções são: aparência, comportamento/interação e estrutura/conteúdo.

2.2 (B) Verdadeiro ou falso? Justifique as falsas.

- JavaScript e Java são a mesma linguagem com nomes diferentes.
- O Node.js permite executar JavaScript fora do navegador.
- Em JavaScript é obrigatório declarar o tipo da variável.
- O código JavaScript de uma página pode alterar o HTML e o CSS dela.

2.3 (B) Qual comando, digitado no terminal, mostra a versão do Node.js instalada? E qual comando executa o arquivo `ola.js`?

2.4 (B) Crie o arquivo `ola.js` que imprime `Olá, mundo!` no terminal e execute-o. Depois altere o programa para imprimir também o seu nome em uma segunda linha.

2.5 (M) Para que serve o Prettier? Configure o VS Code (`settings.json`) para formatar o código automaticamente ao salvar e usar aspas simples. Depois bagunce a indentação de um arquivo, salve, e observe o resultado.

2.6 (M) Diferencie rodar JavaScript no navegador e no Node.js. Dê um exemplo de recurso que existe em um e não no outro (dica: `document`, `window`, `require`).

3. Variáveis, tipos e saída no console

3.1 (B) Declare três variáveis: `nome` (não muda nunca), `idade` (muda a cada aniversário) e `aprovado` (true ou false). Qual palavra-chave você usou em cada uma e por quê? Por que hoje se evita o `var`?

3.2 (B) O que acontece ao executar o código abaixo?

```
const pi = 3.14;
pi = 3.1416;
console.log(pi);
```

3.3 (M) O código abaixo dá erro? O que ele imprime? Explique por que isso não contradiz o fato de `const` não poder ser reatribuído.

```
const frutas = ["maçã"];
frutas.push("uva");
console.log(frutas);
```

3.4 (B) O que imprime cada linha? Qual resultado você achou estranho?

```
console.log(typeof 42);
console.log(typeof "42");
console.log(typeof true);
console.log(typeof undefined);
console.log(typeof null);
console.log(typeof [1, 2]);
console.log(typeof function () {});
console.log(typeof 10n);
```

3.5 (B) O que imprime o código abaixo? Explique a diferença entre `undefined` e `null` e dê um caso em que você atribuiria `null` de propósito.

```
let x;
console.log(x);
```

3.6 (M) Preveja a saída de cada linha.

```
console.log(Number("42") + 1);
console.log("42" + 1);
console.log("42" - 1);
console.log(Number("abc"));
console.log(parseInt("7.9"));
console.log(parseFloat("7.9"));
```

3.7 (M) JavaScript não tem `int` e `float` separados. Quantos tipos numéricos existem na tabela de tipos do slide? Preveja o resultado abaixo e explique por que ele acontece (dica: o mesmo ocorre em Python).

```
console.log(0.1 + 0.2);
console.log(0.1 + 0.2 === 0.3);
```

3.8 (B) Escreva um comentário de linha e um comentário de bloco. Depois comente uma linha de `console.log` para que ela deixe de executar.

3.9 (B) Dadas `const nome = "Maria";` e `const idade = 30;`, imprima `Maria tem 30 anos` de duas formas: passando vários itens ao `console.log` e usando *template literal*. Depois converta para JavaScript este código Python: `print(f"{nome} nasceu em {2025 - idade}")`.

3.10 (M) Escolha o método do `console` mais adequado para cada situação: (a) aviso de que uma função está obsoleta; (b) falha na conexão com o banco; (c) mostrar uma lista de alunos como tabela; (d) limpar o terminal. Depois teste (c) com `[{ nome: "Ana", nota: 9 }, { nome: "Beto", nota: 7 }]`.

4. Operadores e comparações

4.1 (B) Preveja a saída. Depois escreva como obter a divisão inteira de 7 por 2 (que dá 3), já que não existe o operador `//` do Python.

```
console.log(7 / 2);
console.log(7 % 2);
console.log(2 ** 3);
console.log(Math.floor(7 / 2));
```

4.2 (B) Qual é o valor final de `i`? Acompanhe passo a passo.

```
let i = 5;
i++;
i += 2;
i--;
console.log(i);
```

4.3 (M) Preveja cada resultado e explique a diferença entre `==` e `===`. Qual dos dois você usaria no dia a dia?

```
console.log(5 == "5");
console.log(5 === "5");
console.log(5 != "5");
console.log(5 !== "5");
console.log(null == undefined);
console.log(null === undefined);
```

4.4 (B) Preveja a saída dos operadores lógicos.

```
const idade = 20;
const temCarteira = false;
console.log(idade >= 18 && temCarteira);
console.log(idade >= 18 || temCarteira);
console.log(!temCarteira);
```

4.5 (M) Preveja a saída. O que o operador `+` faz quando um dos lados é texto? E o `-` e o `*`?

```
console.log("5" + 3, "5" - 3, "5" * "2");
```

4.6 (D) Preveja a saída e explique por que as duas linhas dão resultados diferentes.

```
console.log(1 + 2 + "3");
console.log("1" + 2 + 3);
```

4.7 (M) Escreva uma expressão booleana que seja `true` apenas quando o número `n` for par **e** positivo. Teste com `n = 4`, `n = -4` e `n = 7`.

5. Condicionais

5.1 (B) Dada `let nota = 7.5;`, escreva um `if / else if / else` que imprima `Aprovado` (nota maior ou igual a 7), `Recuperação` (maior ou igual a 5) ou `Reprovado`. Teste com 9, 7, 6 e 3.

5.2 (M) Dado um número `ano`, imprima se ele é bissexto. Regra: divisível por 4 e não por 100, ou divisível por 400. Teste com 2024, 1900 e 2000.

5.3 (B) Dado `const dia = 3;`, use `switch` para imprimir o nome do dia da semana (1 = domingo ... 7 = sábado) e `Dia inválido` no `default`.

5.4 (M) O que imprime o código abaixo? Por que ele não para no `case 2`? Como corrigir para imprimir apenas `dois`?

```
const n = 2;
switch (n) {
  case 1: console.log("um");
  case 2: console.log("dois");
  case 3: console.log("três");
  default: console.log("fim");
}
```

5.5 (D) Quais letras são impressas? Pesquise os valores *truthy* e *falsy* do JavaScript antes de responder.

```
if ("") console.log("A");
if ("0") console.log("B");
if (0) console.log("C");
if ([]) console.log("D");
if (null) console.log("E");
```

5.6 (D) Dados `peso` e `altura`, calcule o IMC (`peso / altura ** 2`) e classifique: abaixo de 18,5 (abaixo do peso), até 24,9 (normal), até 29,9 (sobrepeso) e acima disso (obesidade). Em seguida, reescreva a classificação da questão 5.1 usando apenas duas opções (`Aprovado` se a nota for maior ou igual a 7, senão `Reprovado`) com o operador ternário. Pesquise a sintaxe.

6. Laços de repetição

6.1 (B) Com `for`, imprima os números de 1 a 10, um por linha.

6.2 (B) Imprima a tabuada do 7 no formato `7 x 1 = 7`, `7 x 2 = 14`, até `7 x 10 = 70`.

6.3 (B) Calcule e imprima a soma de todos os números de 1 a 100, usando uma variável acumuladora. Qual é o resultado esperado?

6.4 (B) Faça uma contagem regressiva de 10 a 1 com `while` e, ao final, imprima `Fogo!`. Depois reescreva o programa com `do...while` e compare.

6.5 (M) Preveja a saída. Qual é o papel de cada uma das três partes entre parênteses?

```
for (let i = 0; i < 10; i += 3) {
  console.log(i);
}
```

6.6 (M) Preveja a saída e explique o que fazem `continue` e `break`.

```
for (let i = 1; i <= 8; i++) {
  if (i % 2 === 0) continue;
  if (i > 6) break;
  console.log(i);
}
```

6.7 (M) Dado `const nomes = ["Ana", "Beto", "Carla"];`, imprima `Olá, Ana!` para cada nome usando `for...of`, sem usar índice. Em seguida, percorra o objeto `{ nome: "Ana", idade: 20 }` com `for...in`, imprimindo `chave: valor`. Qual é a diferença entre `for...of` e `for...in`?

6.8 (D) Calcule o fatorial de 5 (resultado esperado: 120). Depois faça o *FizzBuzz*: de 1 a 30, imprima `Fizz` para múltiplos de 3, `Buzz` para múltiplos de 5, `FizzBuzz` para múltiplos de ambos e o próprio número nos demais casos.

6.9 (D) Usando laços aninhados, imprima este triângulo (5 linhas):

```
*
**
***
****
*****
```

7. Funções

7.1 (B) Escreva a função nomeada `somar(a, b)` que retorna a soma dos dois parâmetros. Chame-a com `2, 3` e imprima o resultado.

7.2 (B) Reescreva a função `somar` de duas outras formas: como função anônima atribuída a uma variável e como *arrow function*. Qual das três você escolheria para uma função de uma linha?

7.3 (B) Preveja a saída das duas chamadas e explique o papel do valor padrão.

```
function saudar(nome = "aluno") {
  return "Olá, " + nome;
}
console.log(saudar());
console.log(saudar("Ana"));
```

7.4 (M) O que acontece em cada um dos dois trechos? Explique por que o primeiro funciona e o segundo não (dica: *hoisting*).

```
// Trecho A
console.log(dobro(4));
function dobro(x) { return x * 2; }

// Trecho B
console.log(triplo(4));
const triplo = function (x) { return x * 3; };
```

7.5 (B) Crie a função `ehPar(n)` que retorna `true` se `n` for par e `false` caso contrário. Teste com 4, 7 e 0.

7.6 (M) O que imprime o código abaixo? Por que a função não devolve a soma? Corrija.

```
function f() {
  1 + 1;
}
console.log(f());
```

7.7 (M) Crie `media(a, b, c)` que retorna a média de três notas com uma casa decimal (ex.: `media(7, 8, 9)` dá `"8.0"`). Que tipo de valor o método `toFixed` devolve?

7.8 (D) Escreva `celsiusParaFahrenheit(c)` (F = C × 9/5 + 32) como *arrow function*. Depois crie uma função `descreverTemperatura(c)` que **usa** a anterior e retorna textos como `"25°C = 77°F"` com *template literal*.

8. Arrays

8.1 (B) Crie um array com cinco frutas. Imprima a primeira, a última (sem fixar o número 4 no código) e o tamanho do array. Depois, percorra o array com `for` e com `for...of`.

8.2 (B) Preveja o conteúdo final de `a`, acompanhando cada linha.

```
const a = [1, 2, 3];
a.push(4);
a.unshift(0);
a.pop();
a.shift();
console.log(a);
```

8.3 (M) O que imprimem as duas linhas? O `slice` altera o array original?

```
const l = [10, 20, 30, 40, 50];
console.log(l.slice(1, 3));
console.log(l);
```

8.4 (B) Com `map`, crie um novo array com o dobro de cada elemento de `[3, 5, 8]`. O array original muda?

8.5 (B) Com `filter`, selecione as notas maiores ou iguais a 7 em `[9, 4.5, 7, 6, 10, 3]`.

8.6 (M) Com `reduce`, some todos os valores de `[10, 25, 40, 5]`. Depois use `reduce` para achar o maior valor do mesmo array.

8.7 (D) Em `const precos = [10, 25, 40, 5];`, aplique 10% de desconto apenas aos preços acima de 20 e some o resultado, encadeando `filter`, `map` e `reduce`. Qual é o valor final?

8.8 (M) O que imprime o código abaixo? Explique por que `x` mudou e como fazer uma cópia independente (dica: operador *spread*, `[...x]`).

```
const x = [1, 2];
const y = x;
y.push(3);
console.log(x);
```

9. Objetos e prototype

9.1 (B) Crie, com a sintaxe literal, um objeto `aluno` com `nome`, `idade` e `notas` (um array com três valores). Imprima o nome usando a notação de ponto, a idade usando colchetes e a segunda nota.

9.2 (B) Preveja a saída. Quais operações foram feitas no objeto?

```
const carro = { marca: "Fiat", ano: 2020 };
carro.cor = "azul";
delete carro.ano;
console.log(carro);
```

9.3 (M) O que imprime o código abaixo? Para que serve a palavra `this` dentro do método?

```
const pessoa = {
  nome: "Ana",
  apresentar() {
    return `Oi, sou ${this.nome}`;
  },
};
console.log(pessoa.apresentar());
```

9.4 (M) Por que a primeira linha imprime `undefined` e a segunda funciona? Quando a notação de colchetes é indispensável?

```
const chave = "nome";
const p = { nome: "Ana", idade: 20 };
console.log(p.chave);
console.log(p[chave]);
```

9.5 (M) Dado `const produto = { nome: "Caneta", preco: 3.5 };`, imprima cada propriedade no formato `nome: Caneta` usando `for...in`. Refaça usando `Object.entries(produto)` com `for...of`.

9.6 (M) Um objeto declarado com `const` pode ter propriedades alteradas, adicionadas ou removidas? E o objeto pode ser trocado por outro (`obj = {}`)? Teste e explique.

9.7 (D) Crie um objeto `conta` com `saldo: 100` e os métodos `depositar(valor)` e `sacar(valor)`. O saque deve ser recusado, com uma mensagem, se o valor for maior que o saldo. Teste com `depositar(50)`, `sacar(30)` e `sacar(500)`.

9.8 (M) Preveja a saída. De onde vem o método `fala` de `cao`? O que diferencia uma propriedade própria de uma herdada?

```
const animal = { fala() { return "..."; } };
const cao = Object.create(animal);
cao.nome = "Rex";
console.log(cao.fala());
console.log(cao.hasOwnProperty("nome"));
console.log(cao.hasOwnProperty("fala"));
console.log(Object.getPrototypeOf(cao) === animal);
```

9.9 (M) O objeto literal `{}` não define nenhum método, mas `({}).hasOwnProperty("a")` e `({}).toString()` funcionam. Explique por quê e confirme com `Object.getPrototypeOf({}) === Object.prototype`. O que vem depois de `Object.prototype` na cadeia?

10. Entrada de dados (prompt-sync e readline)

10.1 (B) Instale o pacote com `npm install prompt-sync` e escreva um programa que pergunta o nome e a idade e imprime `Olá, Ana! Ano que vem você terá 21 anos.` Use `require('prompt-sync')({ sigint: true })` e *template literal*.

10.2 (M) Se o usuário digitar `5` e depois `3`, o que imprime o código abaixo? Por quê? Corrija para que a saída seja `8`.

```
const prompt = require('prompt-sync')({ sigint: true });
const a = prompt('a: ');
const b = prompt('b: ');
console.log(a + b);
```

10.3 (M) Faça uma calculadora: leia dois números e mostre soma, subtração, multiplicação e divisão. Se o segundo número for zero, mostre `Divisão por zero` em vez do resultado da divisão.

10.4 (M) Leia um número e imprima o seu dobro. Se o usuário digitar algo que não seja número, imprima `Valor inválido`. Use `Number.isNaN`. Depois descubra o que `Number("")` devolve quando o usuário só aperta Enter e trate esse caso.

10.5 (M) Teste os dois recursos abaixo e descreva o comportamento de cada um: (a) `prompt('Cidade: ', 'Fortaleza')` quando o usuário apenas aperta Enter; (b) `prompt.hide('Senha: ')` enquanto o usuário digita.

10.6 (M) Reescreva o exercício 10.1 usando o módulo `readline` (`createInterface`, `rl.question` e `rl.close`).

10.7 (D) No código abaixo, qual mensagem aparece primeiro: `Fim do código` ou `Olá, ...`? Explique usando a ideia de *callback* e responda: quando você preferiria `readline` e quando `prompt-sync`?

```
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});
rl.question('Seu nome: ', (nome) => {
  console.log('Olá, ' + nome);
  rl.close();
});
console.log('Fim do código');
```

10.8 (D) Com `prompt-sync`, leia números repetidamente até o usuário digitar `sair`. No final, mostre quantos números foram digitados, a soma e a média.

11. Desafios integradores

Cada desafio combina vários tópicos. Sugestão: faça em duplas, com 15 a 20 minutos por desafio.

11.1 (D) *Adivinhe o número.* O programa sorteia um inteiro de 1 a 100 com `Math.floor(Math.random() * 100) + 1`. O usuário digita palpites (`prompt-sync`) e o programa responde `Maior` ou `Menor` até ele acertar (o programa diz se o número secreto é maior ou menor que o palpite). Ao final, mostre em quantas tentativas ele acertou.

11.2 (D) *Boletim da turma.* Dado o array abaixo, calcule e imprima: (a) a média da turma; (b) o nome de quem foi aprovado (nota maior ou igual a 7); (c) o nome e a nota do aluno com a maior nota. Use `reduce`, `filter` e `map`.

```
const alunos = [
  { nome: "Ana", nota: 9 },
  { nome: "Beto", nota: 5.5 },
  { nome: "Carla", nota: 7 },
  { nome: "Davi", nota: 4 },
];
```

11.3 (M) *Contador de vogais.* Escreva a função `contarVogais(texto)` e teste com `"Desenvolvimento Web"`.

11.4 (D) *Palíndromo.* Escreva `ehPalindromo(texto)`, que ignora maiúsculas e minúsculas. Teste com `"arara"`, `"Ana"` e `"casa"`. Dica: `split`, `reverse` e `join`.

11.5 (D) *Frequência de palavras.* Dada a frase `"a casa é a casa"`, use um objeto para contar quantas vezes cada palavra aparece. O resultado esperado tem a palavra `a` duas vezes, `casa` duas vezes e `é` uma vez.

11.6 (D) *Troco do caixa.* Dado um valor inteiro em reais (por exemplo, 87), diga quantas notas de 50, 20, 10, 5 e 2 são necessárias, usando o menor número de notas. Dica: um array de notas, um laço, `Math.floor` e `%`.